PUC Minas

INFORMAÇÕES SOBRE A AVALIAÇÃO

AV2	**AVALIAÇÃO 2 - 25 pontos**

INFORMAÇÕES DOCENTE						
CURSO:	**DISCIPLINA:**	**TURNO**	**MANHÃ**	**TARDE**	**NOITE**	**PERÍODO/SALA:**
ENGENHARIA DE SOFTWARE	**DIAW**		**x**	**x**	**x**	2°
PROFESSOR (A): João Paulo Carneiro Aramuni						

INFORMAÇÕES DISCENTE	
ALUNO(A):	**RA:**
DATA:	**NOTA:**

INSTRUÇÕES:

1) Consulta:
   I) Será permitido consultar os seus próprios projetos desenvolvidos em laboratório.
   II) Será permitido consultar a documentação oficial do React (react.dev), do Vite (vite.dev), do React Router (reactrouter.com) e do W3C (w3.org) para HTML e CSS.
   III) Não será permitido o uso de inteligência artificial, como o ChatGPT, Claude AI, Gemini, GitHub Copilot, etc., bem como consultas ao Google ou a qualquer outro site de busca.
2) Tecnologias:
   Será obrigatório o uso de Node.js, React, Vite, React Router (react-router-dom), JavaScript (JSX), HTML e CSS.
3) Aparelhos eletrônicos:
   Não será permitido o uso de celulares, relógios digitais ou qualquer outro aparelho eletrônico durante a realização da prova.
4) Conduta:
   O aluno que, durante a prova, utilizar meios ilícitos, como comunicação com outros alunos, consulta a qualquer tipo de material extra prova ou uso de aparelhos eletrônicos, será eliminado do processo avaliativo e receberá nota 0 (zero).
5) Funcionamento:
   Projetos que não rodam, ou seja, que apresentam erro no npm run dev, ou que não possam ser acessados via http://localhost:5173/ serão avaliados com até no máximo 50% da nota da prova (12,5 pts).

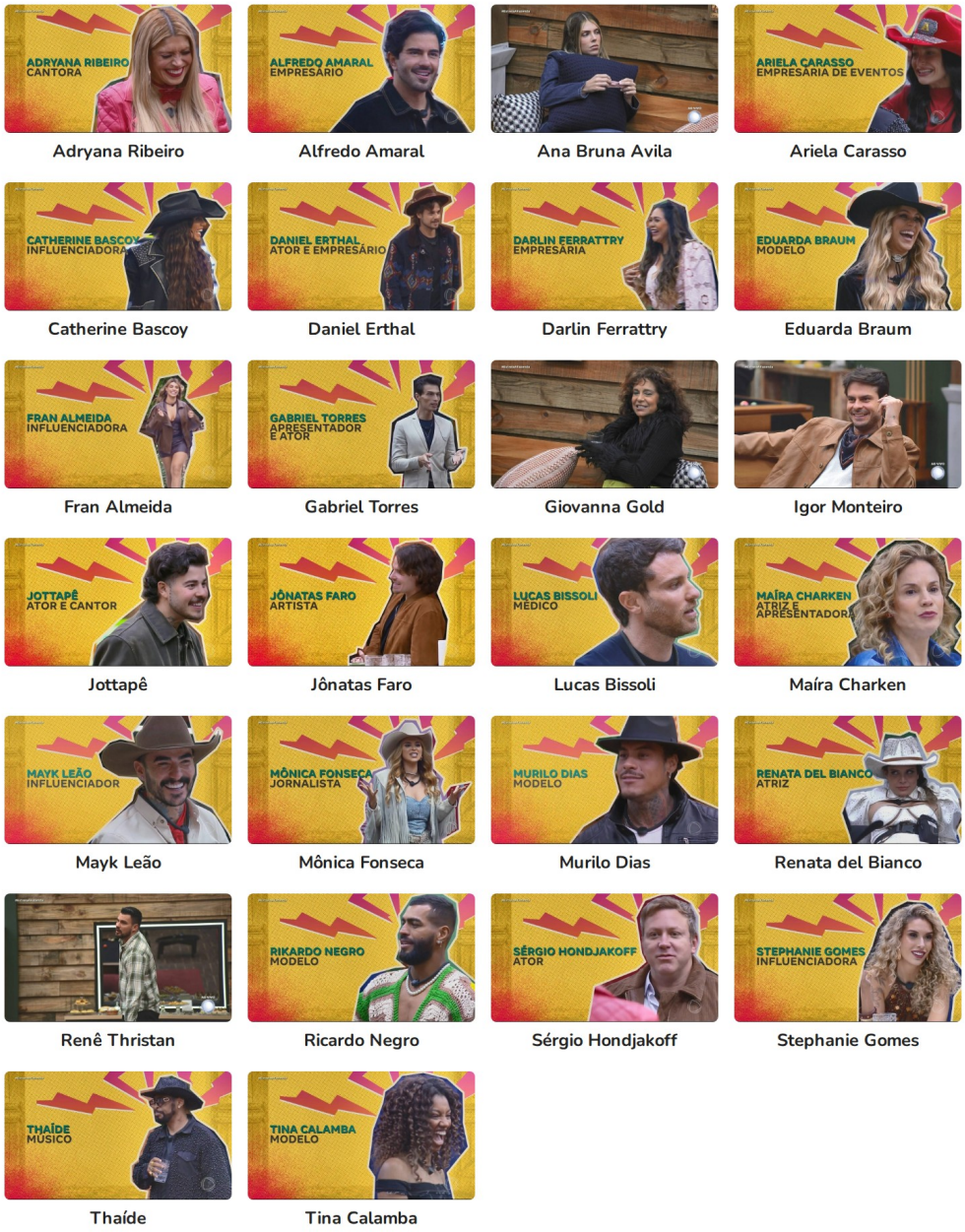
Boa Prova!

Sistema de Votação A Fazenda 2026



O fazendeiro precisa da sua ajuda! Daniel Erthal, Giovanna Gold e Thaíde estão na roça e o público vai decidir quem deixa o programa. Você deve construir, com React e Vite, uma aplicação em que seja possível **votar em um participante da roça para eliminá-lo** e **ver a contabilização total dos votos da eliminação**, com o percentual de cada participante. Os votos devem ficar salvos no navegador (localStorage), usando o service disponibilizado na prova.

Participantes de A Fazenda 2026:



[10,0 pts] Questão 1) Construa os arquivos **components/ParticipanteCard.jsx** e **pages/Votacao.jsx** e realize as chamadas necessárias para os arquivos disponibilizados na prova, chamados services/votacaoService.js e data/participantes.js.

Dicas:

<u>Componente ParticipanteCard</u>

- É um componente de função que recebe por props o objeto participante e, opcionalmente, a função onVotar. Desestruture as props direto na assinatura:

```
function ParticipanteCard({ participante, onVotar }) {
  const { nome, foto, naRoca } = participante;

  return (
    // ...
  );
}

export default ParticipanteCard;
```

- Use o operador ternário no className para destacar quem está na roça e o && para exibir o selo apenas quando naRoca for true:

```
<article className={naRoca ? 'card card-roca' : 'card'}>
  {naRoca && <span className="selo-roca">Na roça</span>}
  <img src={foto} alt={nome} className="card-foto" />
  ...
```

- O botão só deve aparecer quando onVotar for passado. No onClick, use uma arrow function para enviar o participante. Escrever onClick={onVotar(participante)} executaria a função na renderização, e não no clique:

```
{onVotar && (
  <button type="button" onClick={() => onVotar(participante)}>
    Votar para eliminar
  </button>
)}
```

Página Votacao

- Importe do service as funções listarNaRoca(), listarForaDaRoca() e registrarVoto(id). Confira os nomes exatos no arquivo votacaoService.js fornecido.
- Guarde no estado (useState) o último participante que recebeu voto e quantas vezes o usuário já votou desde que abriu a página:

```
const [ultimoVoto, setUltimoVoto] = useState(null);
const [votosNaSessao, setVotosNaSessao] = useState(0);

function handleVotar(participante) {
  registrarVoto(participante.id);
  // atualize os dois estados aqui
}
```

- Renderize os 3 participantes da roça com map(), sempre informando a key, e passe a função handleVotar para o card:

```
{naRoca.map((p) => (
  <ParticipanteCard
    key={p.id}
    participante={p}
    onVotar={handleVotar}
  />
))}
```

- Para os demais peões, reutilize o mesmo ParticipanteCard sem passar onVotar. Assim eles aparecem sem botão.
- Exiba a mensagem de confirmação somente quando ultimoVoto não for null, mostrando:
  - o nome do participante que recebeu o voto;
  - quantas vezes o usuário já votou (cuidado com o singular e o plural: “1 vez”, “2 vezes”);
  - um Link para a página /resultado.

[10,0 pts] Questão 2) Construa os arquivos **App.jsx** e **pages/Resultado.jsx**.

Dicas:
Rotas da aplicação (App.jsx)

- Instale o React Router com o comando npm install react-router-dom.
- Envolva a aplicação com BrowserRouter. O Header fica fora do Routes, para aparecer em todas as páginas. O Header fornecido usa NavLink, que só funciona dentro do BrowserRouter:

```
<BrowserRouter>
  <Header />
  <Routes>
    <Route path="/" element={<Home />} />
    {/* crie aqui as rotas /votacao e /resultado */}
  </Routes>
</BrowserRouter>
```

- Crie as três rotas: / (Home), /votacao (Votacao) e /resultado (Resultado).
- Não se esqueça de importar o arquivo App.css no App.jsx.

Página Resultado

- Inicialize o estado dos votos lendo o localStorage por meio do service. O total e o placar são derivados desse estado, então calcule-os a cada renderização, sem criar outros useState:

```
const [votos, setVotos] = useState(() => obterVotos());

const total = calcularTotalVotos(votos);
const resultado = calcularResultado(votos);
```

- calcularResultado(votos) já devolve a lista ordenada do mais votado para o menos votado, com os campos votos e percentual. O primeiro item (índice 0) é quem sairia hoje: destaque-o com a classe placar-lider e o selo “Sairia hoje”.
- Monte a barra de progresso com style inline, usando o percentual como largura:

```
<div className="barra">
  <div
    className="barra-preenchida"
    style={{ width: `${p.percentual}%` }}
  />
</div>
```

- Formate o percentual no padrão brasileiro com toLocaleString('pt-BR', { maximumFractionDigits: 1 }).
- Quando o total for 0, use o operador ternário para exibir uma mensagem e um Link para /votacao no lugar do placar.
- Para zerar a votação, peça confirmação ao usuário e atualize o estado para que a tela seja renderizada novamente:

```
function handleZerar() {
  if (window.confirm('Apagar todos os votos?')) {
    zerarVotos();
    setVotos({});
  }
}
```

[5,0 pts] Questão 3) Construa o arquivo **App.css**

Dicas:

- Use as variáveis de cor já definidas no index.css fornecido (--celeiro, --palha, --papel, --tinta, --madeira, --pasto) e as fontes --fonte-titulo e --fonte-texto.
- Use CSS Grid para as grades de cards, sem definir manualmente quantas colunas cabem em cada tela:

```
.grade-roca {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 1.5rem;
}
```

- Para a grade dos demais peões, use repeat(auto-fill, minmax(190px, 1fr)).
- Padronize a proporção das fotos com aspect-ratio: 16 / 9 e object-fit: cover (as fotos fornecidas são horizontais).
- Destaque quem está na roça com a classe card-roca (por exemplo, border: 4px solid var(--celeiro)). Para posicionar o selo “Na roça” com position: absolute, o card precisa de position: relative.
- Na barra do resultado, use overflow: hidden no contêiner (.barra) e uma transition na largura de .barra-preenchida, para a barra crescer suavemente.

Observações:
- Crie o projeto com o comando npm create vite@latest VotacaoFazenda -- --template react e depois rode npm install.
- Copie a pasta public fornecida na prova (participantes, fonts, logo-fazenda.jpg e fazendeiro.jpg) para a raiz do projeto.
- Copie os arquivos src/data/participantes.js, src/services/votacaoService.js, src/components/Header.jsx, src/pages/Home.jsx e src/index.css para as mesmas pastas do seu projeto.
- Apague o App.css e a pasta src/assets gerados pelo template do Vite.
- Para conferir ou apagar os votos manualmente, use o DevTools do navegador em Application > Local Storage (chave fazenda2026:votos).
- Use a pasta prints como guia visual do projeto funcionando.
- Leia o arquivo README.md antes de começar a prova.
